Project article: FaceAttend EDU

Jonattan Steven Rizo Solano

Diego Andrés Gutiérrez Núñez

Juan David Arboleda Perdomo

Centro de la Industria, la Empresa y los Servicios, Servicio Nacional de Aprendizaje

Technologist in Software Analysis and Development, Cohort 3145556

November 4, 2026

Project article: FaceAttend EDU

Note to the authors: names, affiliations, and emails are placeholders. Complete them before submission.

Abstract—Recording attendance in technical and higher education is still largely manual. It is slow, and it is easy to falsify with signatures or shared cards. This paper describes FaceAttend EDU, a platform that records attendance with facial recognition and a liveness check. The system has nine backend services behind an API gateway, a React web client, and an Expo mobile client. We describe the architecture, the relational model shared by the services, the biometric subsystem, and the security review we ran before any pilot. We do not report accuracy, latency, or cost figures, because the system has not been measured under field conditions. We report instead what can be checked in the repository: 1,138 commits across all branches between 26 March and 27 October 2026, a 37-table data model, and 2,317 automated tests in the web, mobile, TypeScript, and Java suites. The review found one critical and one serious access-control defect, both open when this text was written. We use the case to discuss three problems that were harder in practice than expected: business rules duplicated across three languages, different route normalization in the gateway and in the services, and documentation that drifted away from the code.

Index Terms—Facial recognition, liveness detection, microservices, attendance management, software architecture, security review, higher education.

Abstract (Spanish version, translated)—Attendance recording in technical and higher education is still mostly manual, slow, and easy to falsify. This article describes FaceAttend EDU, a platform that records attendance through facial recognition with liveness verification. The system consists of nine backend services behind an API gateway, a React web client, and an Expo mobile client. We describe the architecture, the shared relational model, the biometric subsystem, and the security review conducted before any pilot. We do not report accuracy, latency, or cost figures, because the system has not been measured under real usage conditions. We report instead what can be verified in the repository, and we discuss three problems that

turned out to be harder than expected: business rules duplicated across three languages, different route normalization between the gateway and the services, and documentation that drifted away from the code.

I. Introduction

In a class session, recording attendance on paper or in a digital form consumes time that is taken away from the class. Manual recording also allows transcription errors and signatures by someone else, and it is usually reviewed when there is no longer a simple way to correct it.

FaceAttend EDU was created to address this problem in a specific context: a technical training center with several sites, programs organized by cohorts, instructors who open and close sessions, and learners who register in front of a camera. We wanted facial attendance to be an option within a complete academic system, not an isolated module. That forced us to solve identity, roles by site, schedules, justifications, and alerts.

This article does not describe a system validated in production. It describes the design we adopted, the decisions that had a cost, and the results we can support with the code and with automated tests. We contribute three things:

1. A description of the nine-service architecture and the assignment of languages, with its limitations.
2. A 37-table relational model divided into business contexts, with uniqueness rules that the database enforces and not only the application.
3. A pre-pilot security review, with concrete findings, some corrected in a local branch and others open.

Section II places the work in relation to the literature. Sections III to V describe the architecture, the data model, and the biometric subsystem. Section VI summarizes the security review, Section VII reports the automated verification, Section VIII discusses limits, and Section IX concludes. The Appendix indicates how to reproduce the verifications.

II. Related Work

We did not conduct a systematic literature review. This section presents the works we used as a technical basis and those we compared with our decisions.

Facial recognition. FaceNet [1] learns a 128-dimensional Euclidean embedding with triplet loss. ArcFace [2] adds an angular margin to the softmax loss and is frequently used as a baseline in verification. Our biometric service computes 128-dimensional embeddings with the `face_recognition` library [4], which wraps dlib models [3]. The benchmark figures for FaceNet and ArcFace are obtained on public datasets and do not transfer to a classroom, so we do not take them as evidence of our system's performance.

Microservices. Richardson [5] and Newman [6] describe decomposition by business context, asynchronous communication, and eventual consistency as recurring patterns. Several of these patterns, such as the circuit breaker or the saga, appear frequently in the literature and in industry presentations. In our code we did not find a circuit breaker implemented, so we do not describe it as part of the system.

Quality and development practices. Martin [7] argues that clean code reduces long-term maintenance cost. Piñero González et al. [8] address performance-efficiency risks in software products from the early development phases. Both references guided our testing discipline and the decision to measure before optimizing, although in this version of the work we do not systematically measure performance.

III. System Architecture

A. Services and Languages

The backend has nine services and a gateway. Each service owns its data schema and exposes a REST API under `/api/v1`. Table I summarizes the assignment.

Table I. Services and technologies.

Service	Responsibility	Technology
01 identity	People, users, sessions, second factor, recovery	Java, Spring Boot 4.1.x
02 authorization	Roles, permissions, RBAC evaluation	Java, Spring Boot 4.1.x

Service	Responsibility	Technology
03 academic	Sites, programs, cohorts, enrollments	TypeScript, Fastify 5, Drizzle
04 scheduling	Time blocks, class sessions	Java, Spring Boot 4.1.x
05 attendance	Records, justifications, reports	Java, Spring Boot 4.1.x
06 biometric	Enrollment, verification, liveness detection	Python 3.12, FastAPI
07 configuration	Site and security parameters	TypeScript, Fastify 5
08 notification	Alerts and notifications	TypeScript, Fastify 5
09 quality	Project quality tracking	TypeScript, Fastify 5
99 api-gateway	Single entry point, routing, limits	Kong 3.6 [9]

The clients are a web application in React 19 and a mobile application in Expo SDK 57. Both communicate only with the gateway.

We use several languages, and this has a cost that we return to in Section VIII. The Java services rely on Spring Security and Liquibase migrations. The TypeScript services use Fastify [10]. The Python service uses FastAPI [11] and is the only one that depends on the computer vision ecosystem (OpenCV, dlib, and NumPy). The language assignment responds to team decisions; the repository does not justify it in writing beyond what is visible in the code.

B. Communication and Persistence

Each service has its own schema in PostgreSQL 17 [12]. Foreign keys do not cross schemas: when an attendance record references an academic actor, the column stores the identifier and the application validates that it exists. The project documentation sets this rule so that services can migrate separately. The price is that several validations remain outside the database's reach.

Kafka [13] publishes domain events in the Java services and in Academic. MongoDB stores biometric templates, because each record contains a vector and the model version that produced it. Redis stores failed login attempt counters in Identity. Justification support files are stored in MinIO, an S3-compatible service launched with Docker Compose.

C. Internal Organization of a Service

The identity service follows a hexagonal architecture, and the code structure reflects it. The `domain/` package contains the models (`Person`, `User`, `UserSession`), domain events, exceptions, and domain services. The `application/` package contains 34 use-case implementations and input and output ports. The `adapter/` package contains adapters: web and messaging input; output to JPA persistence, security, Kafka, notifications, provisioning, and HTTP clients.

The dependency rule is met in the domain: no file in `domain/` imports `application` or `adapter`. The domain declares 21 events, including `PersonCreated`, `UserAuthenticated`, and `UserLocked`.

D. Decisions with a Cost

Three decisions had visible costs in the repository.

The first was distributing site-scope rules across each service. Because those rules were written in Java, TypeScript, and Python, a fix in one language did not reach the others. Section VI describes a case.

The second was using a declarative gateway. Kong decides which routes are public and which require a session, and each service applies its own rules again. Double verification helps, but it only works if both layers interpret the same route. When they do not interpret it the same way, a gap appears.

The third was declaring a DBML file as the source of truth for the model. This forces derived documentation to be synchronized every time the model changes, and it is a burden the team underestimated. Section IV shows a concrete case of drift.

IV. Data Model and Business Rules

The relational model has 37 tables grouped into contexts: identity, authorization, academic, scheduling, attendance, biometric, configuration, notification, and quality. The central entities are `person`, `app_user`, and `user_session` in identity; `role`, `permission`, and `user_role` in authorization; `cohort`, `enrollment`, and `academic_actor` in academic;

`schedule_block` and `class_session` in scheduling; and `attendance_record` and `justification` in attendance.

Person and user account are separate. A person can have attendance records without having an account, and an account always points to a person. Thus, identity does not depend on the site where a person studies or teaches. The person stores first name, last name, document, email, phone, type and blood group, birth date, address, and status, with audit columns. The account stores the username, password hash, authentication type (`Local`, `Windows`, or `External`), and the mandatory password change and second-factor flags. The hash uses BCrypt with cost 12.

Permissions follow the format `context.resource:action`. The current seeds define four roles (Super Administrator, Administrator, Instructor, and Learner) and 13 permissions. The project's data documentation describes something else: four roles named `SUPER_ADMIN`, `SCHOOL_ADMIN`, `INSTRUCTOR`, and `STUDENT`, and 12 permissions. The code changed and the documentation did not follow. It is the same problem described in Section III.D.

In attendance, the valid states are `Present`, `Absent`, `Late`, and `Justified`, and the capture methods are `FACIAL`, `MANUAL`, `FINGERPRINT`, and `IMPORT`. The attendance rate counts `Present` and `Late` states as attendance. In the attendance code we did not find an automatic lateness rule: the `Late` state is recorded from capture.

Several rules are expressed as database constraints and not only in code:

- **One attendance record per actor and session.** The unique constraint on `(class_session_id, academic_actor_id)` prevents duplicates even if two requests arrive almost simultaneously. Services convert the database error into a conflict response.
- **One justification per record.** Only records in `Absent` or `Late` state are justified, and the unique constraint on `attendance_record_id` prevents a second justification for the same record.
- **Non-overlapping time blocks.** The database imposes uniqueness on the combination of classroom, day, and start time, and on that of instructor, day, and start time. This constraint only catches identical times. Therefore, the scheduling service also validates

that intervals do not cross on the same day, both for the classroom and for the instructor. Blocks that only touch at one endpoint are valid.

- **One session per block and date.** The session state goes from `Open` to `Closed` or `Cancelled`, and closing records who did it and when.

Existence constraints between services cannot be expressed in the database and are validated in the application. This is the most fragile part of the design, and the first we would review in a second version.

V. Facial Recognition and Liveness Verification

The flow has two phases. In enrollment, the biometric service receives an image, verifies that it contains exactly one face, computes the 128-dimensional vector, and stores it in MongoDB along with the model version. If the image has no face, the service reports it; if it has more than one, it rejects the request. In verification, it computes the vector of the captured image and compares it with the person's active template.

The facial comparison uses Euclidean distance, not cosine similarity. According to the service's own configuration comment, dlib vectors are not centered and cosine similarity between different people frequently exceeds 0.85; therefore, the service uses the metric for which the models are trained. The score is `1 - distance`. The default value requires a maximum distance of 0.5 (`face_match_max_distance`). Each site can choose its confidence level on a slider from 50% to 100%. That choice is converted into a maximum distance between 0.60 (50%) and 0.35 (100%). Thus, a site can be stricter than the default value, but it cannot loosen the criterion beyond the 0.6 tolerance that the `face_recognition` library uses by default [4]. The 0.85 cosine threshold in the configuration is not applied to faces: it is used in fingerprint vector verification.

Liveness verification requests actions from a closed list: blink, turn left, turn right, smile, and tilt the head. The service keeps legacy actions for old clients. Each challenge expires after 120 seconds and is signed with a configuration secret. An action answered in less than one second is considered not performed. During the challenge, the service compares each step with the identity reference of the challenge itself using cosine similarity, with a threshold of 0.75, to

check that they belong to the same person. That threshold is looser than the facial comparison threshold because frames taken during a gesture change the vector. This test does not replace an evaluation of presentation attacks with an independent protocol, and we acknowledge this in Section VIII.

We do not report false positive or false negative rates. The repository does not include such a measurement, and we prefer not to publish a figure we could not reproduce.

VI. Security Review

Before any pilot, we reviewed the code by reading the security filters, routers, Kong configuration, and Docker Compose. We also ran the test suites and `npm audit` on the production projects.

Findings fixed in a local branch (unmerged as of the review date):

- **Route-encoding bypass in Node services.** The authentication guard decided whether a route was public using the undecoded URL, but Fastify decodes when routing. A request to `/%61pi/v1/configurations/...` reached the protected handler and the guard treated it as public. Kong was not exploitable via this path: it was tested with `kong:3.6` and forwards the already normalized route. The services were exploitable when called directly on their port. The fix decodes and normalizes before comparing, returns 400 for ambiguous routes, and adds four tests. With real Fastify, the request goes from 200 to 401.
- **Public authorization endpoints.** `GET /api/v1/auth/evaluate` and `GET /api/v1/users/{id}/roles` were marked `permitAll` and Kong routed them. The fix requires a session and, to query another user, the `configuration:manage` permission.
- **Broken Maven wrapper in a clean clone.** `.gitignore` ignored `/.mvn/`, so `mvnw` was versioned but `maven-wrapper.properties` was not. A developer without Maven installed could not build. The fix adjusts `.gitignore` and versions the file.

Open findings:

- A critical access-control defect: a site administrator can obtain a valid session for any user, including the superadministrator, without a password or second factor.
- A serious access-control defect: instructors at one site can read and delete biometric templates of people from other sites.
- The same route-encoding bypass in the Node guard exists in the Java authentication filter, which uses the undecoded route. The local fix does not include it.
- An attendance WebSocket broadcasts events to all sites, not only to the connected user's site.
- Session tokens in `sessionStorage` in the web client.
- Rate limits with development values (`minute: 6000`) on almost all Kong routes.
- Swagger and Actuator accessible without authentication in the four Java services.
- `npm audit` reports 51 vulnerabilities in production dependencies, 34 of them high. The `xlsx` 0.18.5 library has no patch on npm.
- Backup files with secrets (`.env.bak`, `.env.prod.bak`, `.env.prod.bak2`) in the working directory, and a development `.env` in git history. The `.env.prod` file contains the liveness challenge secret. It is not versioned in git, but it is a secret-management risk that must be closed.

None of these findings is resolved in the main branch. We document them because a system that will handle biometric data of minors must record its failures as well as its achievements. Access control and input validation are the areas we recommend auditing first in any similar API [14].

VII. Automated Verification

Table II summarizes the tests run during the review on October 27, 2026. They are unit and integration tests, not end-to-end tests of user stories.

Table II. Automated tests executed.

Component	Suites	Tests	Result
Web client (React)	56	463	all pass

Component	Suites	Tests	Result
Mobile client (Expo)	34	586	all pass
TypeScript services (03, 07, 08, 09)	—	540	all pass, after `npm ci`
Java services (shared: 11; 01: 285; 02: 134; 04: 121; 05: 177)	—	728	all pass
Biometric service (Python)	—	not run	no environment on the review machine

The 2,317 tests executed do not say whether the system works with real learners. They do not measure performance or facial accuracy, and there are no load tests. The end-to-end test directory has 191 lines and covers few flows.

In the internal self-assessment, on a scale from 1 (initial) to 5 (production-ready), the project scores 2.5. MVP scope functionality is rated 3.5. Legal compliance (1.5) and security (2) are the lowest dimensions. This rating is based on reading the code and requirements, not on an evaluation with users.

Two observations from the review seem useful to anyone trying to repeat the work. First: the Maven wrapper did not work in a clean clone, while the Java suite passed in a container with Maven. A test can depend on an environment that no one documented. Second: the bypass in Section VI became visible by reading the code and testing an encoded route; the existing tests did not cover that case.

VIII. Discussion and Limitations

What cost us. Much of the effort was not in facial recognition but in the seams between services. Site-scope rules were duplicated in three languages. According to the internal review report, that duplication explains failures such as cross-site access. If the project were redone, we would centralize those rules in a shared library from the start.

Route normalization is the second case. Two components that interpret the same URL differently create an attack surface that neither sees on its own. In the local branch, the Node guard was fixed; the Java filter has the same pattern and remains open. The lesson is general:

authorization should be evaluated on the route after routing, not on the string that arrived over the network.

What we do not know. We do not know how accurate verification is in a real classroom, with variable lighting, glasses, or low-resolution cameras. Nor do we know how much time attendance recording saves compared with the previous method, because we did not make comparative measurements. A pilot at one site, with a measurement procedure defined before starting, would be the natural next step.

Study limitations. Those of us writing this work also developed the system and conducted its security review. This introduces bias. We mitigated the problem by recording each finding with code evidence, but it does not replace an external audit.

Legal compliance. Biometric data are sensitive data under Article 5 of Law 1581 of 2012 [15]. The same law restricts the processing of minors' data (Article 7), and Decree 1377 of 2013 regulates that processing [16]. As of the review date, the guardian authorization flow was not implemented, nor was the data processing policy or the impact assessment. These are prerequisites for any use with real data.

Licenses and versions. Liquibase 5.x is distributed under an FSL license, which requires an explicit decision before adopting that version. The fingerprint SDK used by the web client did not include its license notices in the review. The Kafka image in Docker Compose is 3.8.0, which according to secondary sources no longer receives community support; its update should be planned.

IX. Conclusions and Future Work

FaceAttend EDU is, for the most part, a functional system. It has a nine-service architecture, a 37-table data model, and 2,317 automated tests that pass. It is not ready for real data. The security review found access-control flaws that must be corrected before a pilot, and legal compliance lags behind the code.

We propose four steps, in this order:

1. Fix and merge the security findings, starting with cross-site access control, with a regression test for each.
2. Centralize site-scope rules in a shared library, with tests generated from a single source.
3. Set up a pre-production environment and measure false positives and false negatives with a valid dataset and consents.
4. Run a pilot at one site, measuring recording time and error before and after with a comparable design.

Appendix. Data and Reproduction

Code. Repository `Jonas7891/ProyectoFaceAttendEDU` (GitHub). The figures in Table II come from the review of October 27, 2026. We did not record with certainty the exact commit of that run. Commit `478785d1` exists in the repository and was the one consulted at the close of the review, but the `tmp/integration` branch no longer exists locally, and the active branch changed afterward (`develop`, commit `bb5f0fb7` at the time of this verification). Before submitting, it is advisable to repeat the tests on the commit that is cited.

Test reproduction (guidance):

- Web client: `npm ci` and then `npm test` in `front-end/Web`.
- Mobile client: `npm ci` and then `npm test` in `front-end/Mobile`.
- TypeScript services: `npm ci` and then `npm test` in `back-end/03-ms-academic`, `07-ms-configuration`, `08-ms-notification`, and `09-ms-quality`.
- Java services: `./mvnw test` in each Java service; if the wrapper is not present, `mvn test` in an image with Maven and JDK 21.
- Biometric service: `poetry install` and `pytest` in `back-end/06-ms-biometric`. It was not run during the review.

Data. The article does not use learner data. The example identifiers in the documentation are fictitious.

Acknowledgments

[Complete: people, institutions, and funding, if applicable.]

References

[1] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 815–823, doi: 10.1109/CVPR.2015.7298682.

[2] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4690–4699.

[3] D. E. King, "Dlib-ml: A machine learning toolkit," *J. Mach. Learn. Res.*, vol. 10, pp. 1755–1758, 2009. [Online]. Available: https://www.jmlr.org/beta/papers/v10/king09a.html

[4] Documentation for `face_recognition`, `compare_faces` function (default tolerance 0.6). [Online]. Available: https://face-recognition.readthedocs.io/en/stable/face_recognition.html

[5] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY, USA: Manning, 2018.

[6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.

[7] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ, USA: Prentice Hall, 2008.

[8] M. Piñero González, A. Marín Díaz, Y. Trujillo Casañola, and D. Buedo Hidalgo, "Good practices to prevent performance-efficiency risks in software products," *Rev. Cubana de Ciencias Informáticas*, vol. 15, no. 1, pp. 89–113, 2021.

[9] Kong Inc., *Kong Gateway Documentation*, version 3.6. [Online]. Available: https://docs.konghq.com/

[10] Fastify, *Fastify Documentation*, version 5. [Online]. Available: https://fastify.dev/docs/latest/

[11] FastAPI, *FastAPI Documentation*, version 0.162. [Online]. Available: https://fastapi.tiangolo.com/

[12] PostgreSQL Global Development Group, *PostgreSQL 17 Documentation*. [Online]. Available: https://www.postgresql.org/docs/17/

[13] Apache Software Foundation, *Apache Kafka Documentation*, version 3.8. [Online]. Available: https://kafka.apache.org/38/apis/

[14] OWASP Foundation, *OWASP API Security Top 10 — 2023*. [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0x11-t10/

[15] Congress of Colombia, *Statutory Law 1581 of 2012, which establishes general provisions for the protection of personal data*, Diario Oficial No. 48.587, Bogotá, Colombia, Oct. 2012. [Online]. Available: https://www.unidadvictimas.gov.co/wp-content/uploads/Documentos/ley/Ley_1581_2012.pdf

[16] Presidency of the Republic of Colombia, *Decree 1377 of 2013, which partially regulates Law 1581 of 2012*. [Online]. Available: https://www.unidadvictimas.gov.co/wp-content/uploads/Documentos/decretos/DECRETO_1377_2013.pdf